README - robot arm

CPS TEAM 17
Andrea Spagnolo, Bianca Zhong, Zhiru Niu

1. Project Description

This project uses webcam input and machine learning to Arduino to control a physical system with four servo motors.

A Processing sketch captures video from the webcam and calculates the frame-area ratios for red, blue, green, and orange. It sends these 4 values to a trained Wekinator model over OSC. Wekinator is configured with 4 inputs, 1 output, and 4 classes: red, blue, green, and orange.

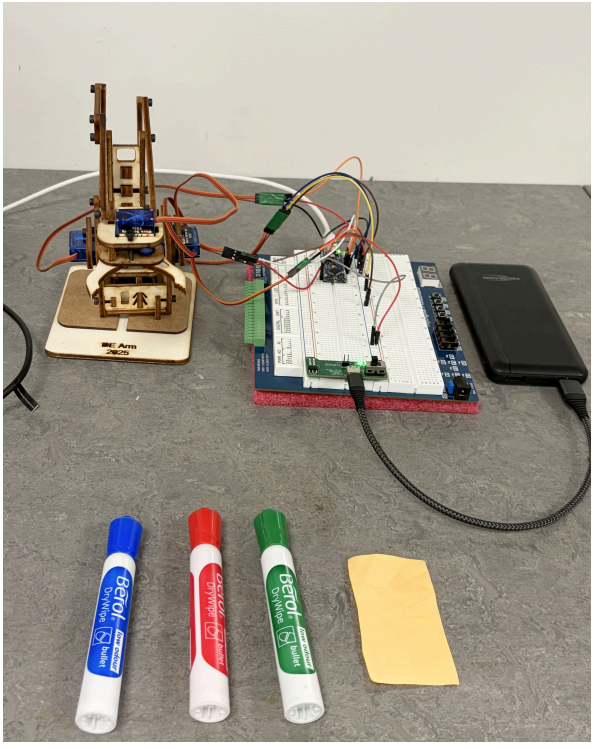
Wekinator sends its predicted class to Processing using OSC. Processing checks whether the corresponding colour occupies more than one-third of the camera frame. If the condition is satisfied, Processing sends the class number to the Arduino over USB serial, where each colour controls a separate servo.

2. System Setup

2.1 Required Hardware and Software

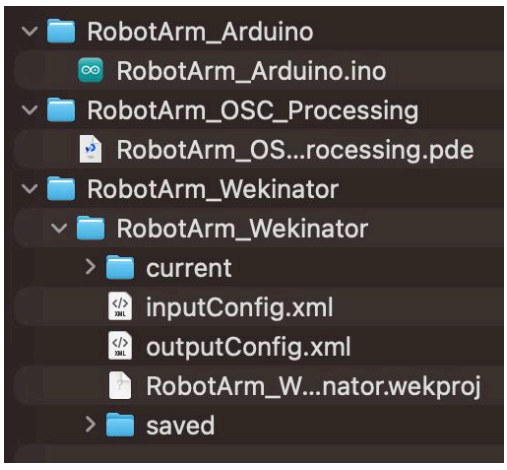
1 × Robot arm with four servos
1 × Arduino Nano Every
1 × Breadboard
1 × Power supply board
1 × USB power bank
1 × laptop with built-in or external camera (not shown)

USB cables and jumper wires
Red, blue and green markers, and orange paper for colour input

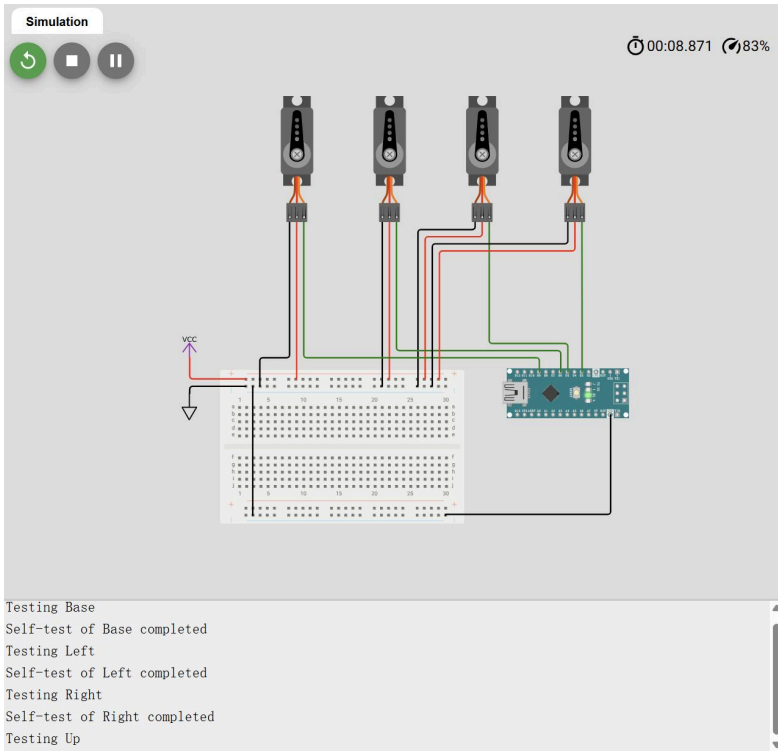


Go to (https://github.com/skdpla24/CPS-Group-17-Robot-Arm-Program) download file

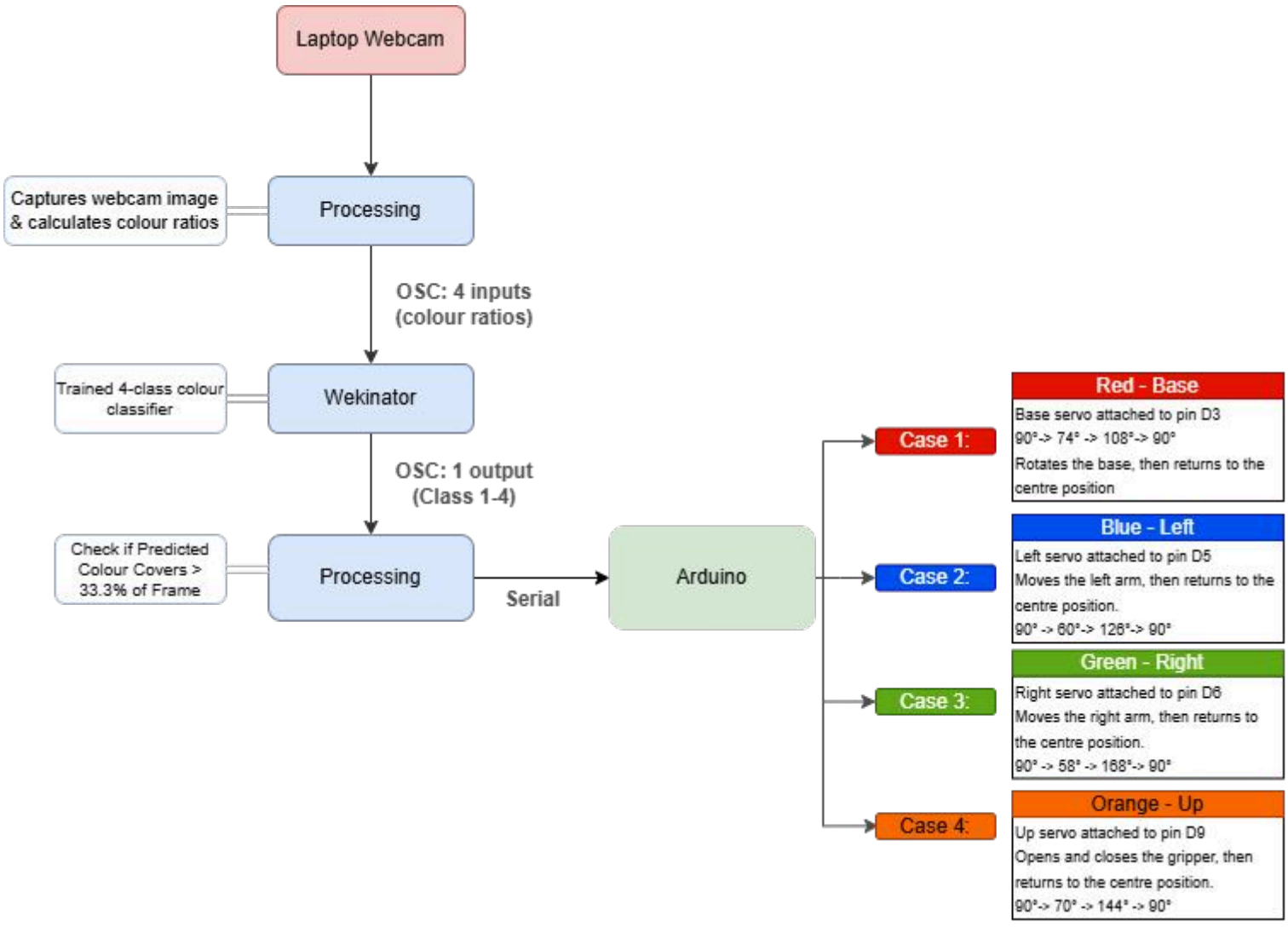
Extract Robot Arm Program.zip. The image below shows all the program files used in this project.



2.2 Wiring Diagram



3. System Architecture



4. Testing and Validation

4.1 First run

This unit test is done to verify that all four servos are connected to the correct signal pins and can perform their programmed movements independently of the webcam, Processing and Wekinator.

a. Connect the hardware. Connect servo signal wires (Pin location sees: https://content.arduino.cc/assets/Pinout-NANOevery_latest.png) to D3 (Base), D5 (Left), D6 (Right), and D9 (Up). Use a suitable external supply for the servos and connect its ground to Arduino GND. Do not power four servos from the Arduino board's 5 V pin.

b. Upload the Arduino sketch. Open either RobotArm_Arduino.ino sketch in Arduino IDE, select the correct board and port, and upload it. The servos run their self-test at startup.
Tips: The Processing sketch currently contains a macOS serial path:
java
port = new Serial(this, "/dev/cu.usbmodem1201", 9600);
On Windows, replace it with the Arduino's current COM port, for example:
java
port = new Serial(this, "COM8", 9600).

c. Free the serial port. Close Arduino IDE Serial Monitor and any other program using the Arduino port.

d. Open Wekinator. Click-File-Open Project-Open RobotArm_Wekinator.wekproj. Confirm 4 inputs, 1 classification output, 4 classes, input port 6448, and output port 12000. To retrain, start Processing so feature data is arriving, record examples for the four class labels, train the classifier, and save the model. Existing training data and a model are in current.

e. Configure and run Processing. Install the Processing Video and oscP5 libraries. In RobotArm_OSC_Processing.pde, set the serial port to the Arduino's current port, then run the sketch. Check the camera preview and console output.

f. Interact. Keep Wekinator running. Place one colour in the camera view so it covers more than one-third of the frame. Confirm the predicted class and area ratio, then observe the corresponding servo move and return.

4.2 Webcam colour analysis and Wekinator communication

Unit test done to verify that Processing extracts the four colour ratios and that Wekinator receives the inputs and produces a class prediction.

a. Start Wekinator and load the trained four-class classification model.
b. Start the Processing application and confirm that the webcam image is visible.
c. Present red, blue, green and orange objects to the webcam one at a time.
d. Observe the colour percentages displayed by Processing.
e. Confirm that the percentages change when the visible colours change.
f. Check that Wekinator receives the OSC inputs and returns a class output.

Processing calculates the proportion of the camera frame classified as each colour and sends four floating-point inputs through the OSC address /wek/inputs.

5. Debugging Guide

5.1 Serial port not found in Processing

Check the port name in this line:
port = new Serial(this, "/dev/cu.usbmodem1201", 9600);
The name must match the Arduino port on your computer. macOS commonly uses /dev/cu.usbmodem..., while Windows uses names such as COM3 or COM8. Check the available ports printed in the Processing console.

5.2 “Port busy” error in Processing

If Processing displays:
Error opening serial port /dev/cu.usbmodem21201: Port busy
Close the Arduino Serial Monitor and Serial Plotter, then run Processing again. Another application may already be using the port.

5.3 Wekinator receives no inputs or produces no class

- Check the Wekinator project is open and configured for 4 inputs, one classification output, and 4 classes.
- Confirm Processing is running and sending `/wek/inputs` to 127.0.0.1:6448.
- Confirm Wekinator sends /wek/outputs to localhost:12000 and Processing listens on port 12000.
- Check that training examples use the same class mapping as the table above and that the classifier has been trained (or the saved model loaded).

5.4 Self-test messages appear, but a servo does not move

- Check that the external servo power supply is switched on and that Arduino and the servo supply share a common ground.
- Switch off the servo power before checking loose connections
- Repeat the self-test and observe each servo. Completion messages only confirm that the code sequence has finished; they do not confirm physical movement.

5.5 Wekinator predicts a class, but the arm does not respond

- Check the Processing console for “Received Wekinator class”. If this message is missing, check the OSC settings in Section 5.6.
- Check the matching colour percentage in the camera window. It must exceed one-third of the frame before Processing sends a command.

5.6 The same colour does not trigger another movement

- Processing prevents the same active class from being sent repeatedly.
- Remove the colour and wait until the Processing window displays “Class: -1”, then present it again.
- Keep Wekinator running during this process so Processing can receive predictions and reset the trigger.

5.7 Movements repeat, respond late, or continue after the colour is removed

- During our testing, leaving a colour in front of the camera for too long sometimes caused repeated movements. Changes in the predicted class or fluctuations around the area threshold may trigger additional commands.
- Each Arduino action contains approximately 1.5 seconds of delays. Commands received during this time may queue.
- Present the colour until a movement is triggered, then remove it quickly. Wait for the movement to finish before triggering another.

5.8 A servo buzzes continuously or the mechanism gets stuck

- Switch off the external servo power and support the arm if needed.
- Check for obstructions, loose parts, or a joint pushing against a mechanical limit.
- After correcting the problem, repeat the Arduino-only self-test before restarting the full system.